PySpark DataFrame Operations

Coding Assignment — Module 2

20 Hands-On Questions | Select • Transforms • Rename • Duplicates • Nulls

Course	Module	Trainer	Scoring
PySpark for Data Engineering	DataFrame Operations — Module 2	Sateesh (MCT, 22+ yrs exp)	Easy 1pt · Medium 2pt · Hard 3pt

Assignment Instructions

- Use the provided dataset for all questions unless a question specifies otherwise.
- Write clean, executable PySpark code. Initialize SparkSession once at the top.
- Use .show() to display output and .printSchema() where asked.
- Submit your Colab/Jupyter notebook with all cells executed and outputs visible.
- Easy = 1 pt | Medium = 2 pts | Hard = 3 pts | Bonus = 5 pts | Total = 42 pts

Sateesh Says
Module 2 is where the real work begins. select, withColumn, filter — these are the 'bread and butter' of every Databricks pipeline at Infosys, TCS, and Wipro. Master these and you can transform any dataset in the world!

Dataset — Use This for All Questions

Dataset 1: E-Commerce Orders (orders_data)

orders_data — Preview

order_id	customer_name	city	product	category	quantity	unit_price	order_date	is_delivered
1001	Arjun Sharma	Hyderabad	Laptop	Electronics	1	75000	2024-01-10	true
1002	Priya Reddy	Bangalore	Phone	Electronics	2	25000	2024-01-15	true

1003	Vikram Nair	Mumbai	Desk	Furniture	1	12000	2024-02-01	false
1004	Neha Gupta	Delhi	Chair	Furniture	4	8000	2024-02-14	true
1005	Rahul Iyer	Hyderabad	Laptop	Electronics	1	75000	2024-03-05	false
1006	Ananya Patel	Pune	Headphones	Electronics	3	5000	2024-03-20	true
1007	Karthik Rao	Bangalore	Phone	Electronics	2	25000	2024-04-02	true
1008	Deepika Singh	Chennai	Monitor	Electronics	2	18000	2024-04-18	false
1009	Suresh Kumar	Hyderabad	Chair	Furniture	2	8000	2024-05-01	true
1010	Meera Joshi	Mumbai	Desk	Furniture	1	12000	2024-05-15	true
1011	Arjun Sharma	Hyderabad	Laptop	Electronics	1	75000	2024-06-01	true
1012	Priya Reddy	Bangalore	Phone	Electronics	2	25000	2024-06-10	false

```
from pyspark.sql import SparkSession, Row
import datetime

spark = SparkSession.builder.appName('LevelUpModule2').getOrCreate()

orders_data = [
  (1001,'Arjun Sharma', 'Hyderabad','Laptop',
'Electronics',1,75000,datetime.date(2024,1,10), True ),
  (1002,'Priya Reddy',  'Bangalore','Phone',
'Electronics',2,25000,datetime.date(2024,1,15), True ),
  (1003,'Vikram Nair',  'Mumbai',  'Desk',        'Furniture',
1,12000,datetime.date(2024,2, 1), False),
  (1004,'Neha Gupta',   'Delhi',   'Chair',       'Furniture',  4,
8000,datetime.date(2024,2,14), True ),
  (1005,'Rahul Iyer',   'Hyderabad','Laptop',
'Electronics',1,75000,datetime.date(2024,3, 5), False),
  (1006,'Ananya Patel', 'Pune',    'Headphones',  'Electronics',3,
5000,datetime.date(2024,3,20), True ),
  (1007,'Karthik Rao',  'Bangalore','Phone',
'Electronics',2,25000,datetime.date(2024,4, 2), True ),
  (1008,'Deepika Singh','Chennai',  'Monitor',
'Electronics',2,18000,datetime.date(2024,4,18), False),
  (1009,'Suresh Kumar', 'Hyderabad','Chair',      'Furniture',  2,
8000,datetime.date(2024,5, 1), True ),
  (1010,'Meera Joshi',  'Mumbai',   'Desk',       'Furniture',
1,12000,datetime.date(2024,5,15), True ),
```

```
  (1011,'Arjun Sharma', 'Hyderabad','Laptop',
'Electronics',1,75000,datetime.date(2024,6, 1), True ),
  (1012,'Priya Reddy',  'Bangalore','Phone',
'Electronics',2,25000,datetime.date(2024,6,10), False),
]

order_cols = ['order_id','customer_name','city','product','category',
              'quantity','unit_price','order_date','is_delivered']

orders_df = spark.createDataFrame(orders_data, order_cols)
```

Dataset 2: Customers with Nulls (for na.drop / dropDuplicates)

```
customers_data = [
  (1, 'Ravi Verma',  'Mumbai',    29, 'ravi@email.com'),
  (2, 'Sita Rao',    'Delhi',    None, None           ),
  (3,  None,         'Hyderabad', 30, 'anon@email.com'),
  (4, 'Kiran Mehta', None,       None, None           ),
  (5, 'Ravi Verma',  'Mumbai',    29, 'ravi@email.com'),
  (6, 'Amit Singh',  'Pune',      35, 'amit@email.com'),
  (7, 'Sita Rao',    'Delhi',    None, None           ),
]
cust_cols = ['cust_id','name','city','age','email']
customers_df = spark.createDataFrame(customers_data, cust_cols)
```

□ Assignment Questions

Section 1: Select & Projections [Easy]

Q1. **[Easy]** | *select — Basic Columns*

Using orders_df, select and display only the following columns: order_id, customer_name, product, unit_price.
Try all three approaches:
(a) Using column name strings: df.select('col1','col2',...)
(b) Using a list: df.select(['col1','col2',...])
(c) Using col() function: df.select(col('col1'),...)

Q2. **[Easy]** | *select('*') and Alias*

Assign an alias 'o' to orders_df using .alias(). Then use select('o.*') to display all columns. Also demonstrate selecting specific columns using 'o.customer_name' style notation. Show both results.

Q3. **[Easy]** | *selectExpr*

Use selectExpr() on orders_df to:
(a) Select order_id, customer_name, city
(b) Create a calculated column: unit_price * quantity named as total_value (write it as a SQL expression string)
(c) Show the result.

Q4. **[Easy]** | *Spark SQL via Temp View*

Register orders_df as a temporary view called 'orders'. Then write a spark.sql() query to: select order_id, customer_name, city, product, unit_price from orders where category = 'Electronics'. Show the result.

Sateesh Says
Flipkart's data team uses selectExpr every day to compute cart totals inline — just like Q3. Knowing col(), select(), and spark.sql() gives you 3 weapons to write the same logic. Pick the cleanest for each situation.

Section 2: col(), concat(), lit() [Easy–Medium]

Q5. **[Easy]** | *col() and concat()*

Using orders_df, create a new column 'customer_product' by concatenating customer_name, a literal ' | ', and product using concat() and lit().
Display: order_id, customer_product, unit_price.
Use .alias() to name the concatenated expression.

Q6. **[Easy]** | *lit() — Adding Constant Columns*

Add two constant columns to orders_df:
(a) 'platform' with value 'LevelUp Store'
(b) 'discount_pct' with value 10 (integer)
Use lit() inside withColumn(). Show the final DataFrame with all original columns plus the two new ones.

Q7. **[Medium]** | *cast() — Type Conversion*

From orders_df:
(a) Cast unit_price (currently LongType) to DoubleType — name it 'unit_price_double'
(b) Cast order_date to StringType — name it 'order_date_str'
(c) Cast is_delivered (BooleanType) to IntegerType — name it 'delivered_flag'
Use withColumn() for each. Show the DataFrame and print the schema.

Q8. **[Medium]** | *date_format() + cast()*

Using orders_df, select order_id and order_date.
(a) Format order_date as 'yyyyMMdd' string using date_format()
(b) Then cast that formatted string to IntegerType
(c) Alias the result as 'order_date_int'
Hint: from pyspark.sql.functions import date_format
Show the result and print the schema.

Sateesh Says
cast() is like currency conversion at an airport. Swiggy stores timestamps as strings in their Kafka events, then casts to TimestampType in Spark before aggregating. Without cast(), your date filters silently fail.

Section 3: Column Operations — withColumn & Rename [Medium]

Q9. **[Medium]** | *withColumn() — Add & Modify*

Using orders_df:
(a) Add a new column 'total_value' = quantity * unit_price
(b) Add a new column 'discounted_price' = unit_price * 0.90 (10% off)
(c) Replace the 'unit_price' column with unit_price + 500 (shipping charge)
Use withColumn() for each step. Show the final DataFrame.

Q10. **[Medium]** | *alias() in select()*

Using orders_df, use .select() with .alias() to rename columns in the output:
- order_id → 'ID'
- customer_name → 'Customer'
- product → 'Item'
- unit_price → 'Price (INR)'
- city → 'Location'

Display only these 5 renamed columns.

Q11. **[Medium]** | *withColumnRenamed()*

Using orders_df:
(a) Rename 'customer_name' to 'buyer_name' using withColumnRenamed()
(b) Then rename 'unit_price' to 'price_inr' using withColumnRenamed()
(c) Rename 'is_delivered' to 'delivered' in a third step
Chain all three withColumnRenamed() calls. Show the result.

Q12. **[Medium]** | *toDF() — Rename Multiple Columns*

Create a small DataFrame:
data = [('TCS','IT Services',2000000), ('Infosys','IT Services',350000), ('Wipro','IT',250000)]
columns = ['company','sector','headcount']
Use .toDF('Company Name','Industry','Total Employees') to rename all 3 columns at once. Show result.

Q13. **[Medium]** | *drop() — Remove Columns*

Using orders_df:
(a) Drop the 'order_date' column using .drop()
(b) Create another version dropping both 'category' and 'is_delivered' in one .drop() call
Show both results and confirm the dropped columns are gone.

Sateesh Says
In Databricks at Cognizant, a pipeline typically does: read raw → withColumn to add 3 computed cols → withColumnRenamed for clean naming → drop internal metadata cols → write to Silver layer. That's Q9-Q13 in sequence!

Section 4: Narrow vs Wide Transformations [Medium–Hard]

Q14. **[Medium]** | *Identify Transformation Type*

Look at the following operations on orders_df. For each one, state whether it is a Narrow or Wide transformation and explain why in a comment:
(a) orders_df.select('order_id','product')
(b) orders_df.filter(col('city') == 'Hyderabad')
(c) orders_df.groupBy('category').count()
(d) orders_df.distinct()
(e) orders_df.withColumn('total', col('quantity')*col('unit_price'))
Write the code and add a comment explanation for each.

Q15. **[Hard]** | *asc() and desc() Sorting*

Using orders_df:
(a) Sort by unit_price ascending using .orderBy(col('unit_price').asc())
(b) Sort by unit_price descending using .orderBy(col('unit_price').desc())
(c) Sort by category ascending, then unit_price descending (multi-column sort)
Show all three results. State in a comment: is sort/orderBy a Narrow or Wide transformation?

Q16. **[Medium]** | *contains() — Filter with Column Function*

Using orders_df, use the .contains() column function to:
(a) Filter rows where customer_name contains 'Sharma'
(b) Filter rows where city contains 'bad' (will match Hyderabad, Ahmedabad)
(c) Filter rows where product contains 'o' (Phone, Monitor, Headphones)
Show results for each filter. Use col('column').contains('value') syntax.

Section 5: Duplicates & Null Handling [Medium–Hard]

Q17. **[Medium]** | *count(), distinct(), dropDuplicates()*

Using customers_df:
(a) Get total row count using .count()
(b) Get distinct row count using .distinct().count()
(c) Use .dropDuplicates(['name','city']) to remove duplicates based on those two columns
(d) Show the deduplicated DataFrame.
Compare (a) and (b) — how many duplicates exist?

Q18. **[Medium]** | *na.drop() — Remove Null Rows*

Using customers_df:
(a) Drop all rows where ANY column has a null: df.na.drop(how='any') — show result
(b) Drop rows where ALL columns are null: df.na.drop(how='all') — show result
(c) Also demonstrate: df.dropna(how='any') as equivalent syntax
Compare the row counts for (a) and (b) and add a comment explaining the difference.

Q19. **[Hard]** | *Full Transformation Pipeline*

Using orders_df, build a complete pipeline in sequence:
Step 1: Add 'total_value' = quantity * unit_price using withColumn()
Step 2: Add 'platform' = 'Flipkart' as a constant using lit()
Step 3: Rename 'customer_name' to 'buyer' using withColumnRenamed()
Step 4: Drop 'order_date' and 'is_delivered' columns
Step 5: Filter only rows where category = 'Electronics' using .filter()
Step 6: Sort by total_value descending
Chain all steps. Show the final DataFrame.

Sateesh Says
Q19 is a mini Silver-layer pipeline — exactly what Zomato runs in Databricks every 15 minutes for their order analytics. Raw orders come in, get enriched, cleaned, filtered, sorted, and written to Delta Lake. You just built that!

Q20. **[Hard]** | *selectExpr + concat + cast + SQL Combined*

Using orders_df, write ONE selectExpr() call that produces the following columns:
(a) order_id
(b) Full label: 'Order #' concatenated with CAST(order_id AS STRING) → alias 'order_label'
(c) total_value = quantity * unit_price → alias 'total_value'
(d) unit_price formatted as string with 'INR ' prefix → alias 'price_display'
(e) order_date cast to string → alias 'order_date_str'
Show the result. Use pure SQL expression strings inside selectExpr().

BONUS QUESTION [+5 Points]

Mini Project: LevelUp Course Sales Report Pipeline

Create a fresh dataset of 8 LevelUp course enrollment records with columns:
enrollment_id, student_name, city, course_name, fees_paid, enrolled_date, is_certified, rating

1. Create the DataFrame with an explicit StructType schema.
2. Add column 'fees_with_gst' = fees_paid * 1.18 using withColumn().
3. Add column 'source' = 'LevelUp Website' as a constant using lit().
4. Rename 'student_name' to 'learner' and 'fees_paid' to 'course_fee'.
5. Drop the 'enrolled_date' column.
6. Use selectExpr to create: enrollment_id, learner, city, course_name, ROUND(fees_with_gst,2) as 'total_fees', is_certified.
7. Filter only certified students (is_certified = True), sort by total_fees descending.
8. Register as a temp view 'certified_learners' and run a spark.sql() query to show count per city.

Scoring Rubric

Difficulty	Questions	Points Each	Question Nos.	Total
Easy	6	1 pt	Q1, Q2, Q3, Q4, Q5, Q6	6 pts
Medium	10	2 pts	Q7–Q14, Q16, Q17, Q18	20 pts
Hard	4	3 pts	Q15, Q19, Q20 + 1 bonus	12 pts
Bonus	1	5 pts	Mini Project	5 pts

TOTAL	**21**	**—**	**—**	**43 pts**

Sateesh Says
If you can complete Q19 and Q20 without any hints, you are ready for a PySpark Data Engineer interview at any top Indian IT company. Q19 is literally a Bronze→Silver pipeline. Q20 is what Databricks notebooks look like in production!

All the Best! — Sateesh | LevelUp Cloud Trainings
www.levelupedu.net | Hyderabad, India